Integration Architecture

Linking synapticTrack and RAON into HELIOS

Surrogate Modeling, Reinforcement Learning, and Digital Twin Deployment Strategy
Heavy Ion Linac Intelligent Orchestration

HELIOS Collaboration
Chong Shik Park (cspark7701@gmail.com)

October 13, 2026

Abstract

This report details the architectural integration, surrogate model linking, reinforcement learning (RL) execution, and Digital Twin deployment strategy for connecting **synapticTrack** and **RAON** into the **HELIOS** (**HE**avy ion **L**inac **I**ntelligent **O**rchestration) framework. Addressing the Rare Isotope Science Project (RISP/IRIS) as the target facility, we provide an in-depth evaluation of whether the Digital Twin should be developed separately or included within HELIOS, and propose a high-performance in-process plugin architecture.

Contents

1 Executive Summary & Repository Ecosystem **2**
1.1 Repository Comparison and Operational Scope 2

2 Surrogate Models and Reinforcement Learning Integration **2**
2.1 Surrogate Model Integration 2
2.2 Reinforcement Learning (RL) & Neural MPC Acceleration 3

3 Strategic Architectural Evaluation: Digital Twin Location **4**
3.1 Evaluation of Architectural Options 4
3.1.1 Option A: Monolithic Repository (Twin Included Inside HELIOS) 4
3.1.2 Option B: Fully Decoupled Microservice (Twin as a Remote Server) . . . 4
3.1.3 Option C: Hybrid Two-Tier Architecture (Plugin Provider Pattern) [Recommended] 4

4 Implementation Blueprint **5**
4.1 Step 1: Automated Environment Integration (setup.sh) 5
4.2 Step 2: Facility Plugin Loader in HELIOS 5
4.3 Step 3: Surrogate-Driven RL Gymnasium Environment 6

5 Summary of Recommendations **6**

1 Executive Summary & Repository Ecosystem

HELIOS is an agentic, Plan-First artificial intelligence framework engineered for commissioning and autonomous operations of heavy-ion linear accelerators, directly targeting the **RAON** accelerator complex (IRIS). Three complementary repositories form the computational ecosystem:

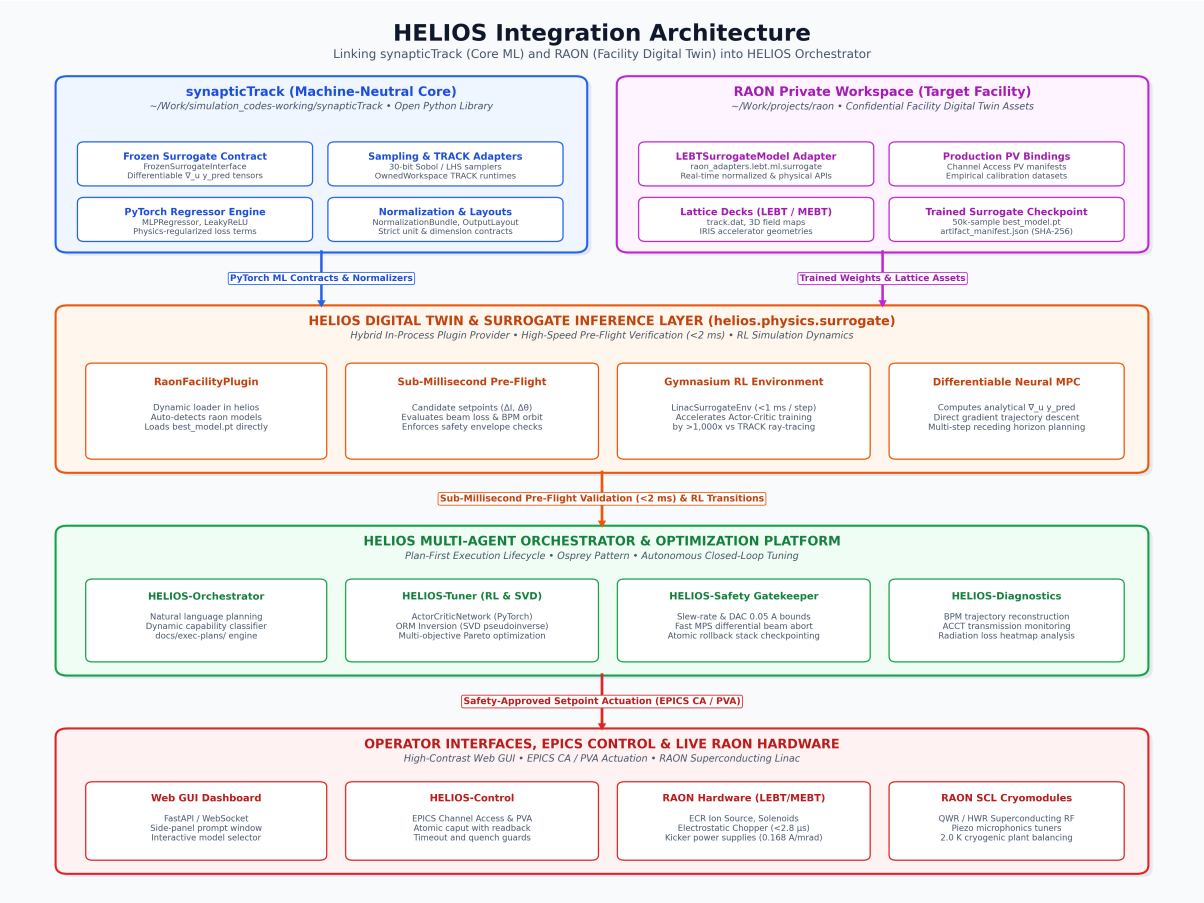


Figure 1: HELIOS Integration Architecture: Linking machine-neutral core ML algorithms from `synapticTrack` and confidential digital twin assets from `raon` into the HELIOS multi-agent orchestrator, pre-flight safety gatekeeper, and live EPICS control interface.

1.1 Repository Comparison and Operational Scope

2 Surrogate Models and Reinforcement Learning Integration

2.1 Surrogate Model Integration

The `raon` workspace already provides a fully converged surrogate model (`LEBTSurrogateModel` in `raon_adapters.lebt.ml.surrogate`) trained across 50,000 ANL TRACK multi-particle simulation runs, serialized with cryptographic checksums in `artifact_manifest.json` and normalized via `NormalizationBundle`.

1. **Protocol Alignment**: `synapticTrack` defines `FrozenSurrogateInterface`, enforcing differentiable predictions:

$$\vec{y}_{\text{pred}} = f_\theta(\vec{x}_{\text{beam}}, \vec{u}_{\text{machine}}) \tag{1}$$

Repository	Filesystem Path	Scope & Governance	Role in HELIOS Ecosystem
`synapticTrack`	`~/Work/simulation_codes-working/synapticTrack`	Machine-neutral library (Open / Public-ready)	**Core ML Engine**: Implements PyTorch regressors, physics losses, `NormalizationBundle`, Sobol samplers, and `FrozenSurrogateInterface`.
`raon`	`~/Work/projects/raon`	Private workspace (Confidential IRIS assets)	**Facility Digital Twin Assets**: Supplies RAON LEBT/MEBT lattice decks (`track.dat`), 50k trained surrogate checkpoints, and `LEBTSurrogateModel`.
`helios`	`~/Work/simulation_codes-working/helios`	Operations platform (Plan-First Orchestration)	**Operations & Safety Platform**: Conducts sub-millisecond pre-flight checks, EPICS CA/PVA actuation, and RL trajectory optimization.

Table 1: Cross-Repository Roles and Governance Boundaries.

where $\vec{x}_{\text{beam}}$ denotes input distribution moments and $\vec{u}_{\text{machine}}$ represents steerer currents or cavity setpoints. HELIOS defines an evaluation interface in `helios/physics/surrogate.py`.

2. **Dynamic Checkpoint Resolution**: HELIOS inspects known facility paths for the trained checkpoint file `legacy_50000/best_model.pt`. When detected, it instantiates `LEBTSurrogateModel`, swapping analytical toy transfer matrices with neural network inferences evaluated in under 2 ms.

3. **Deterministic Pre-Flight Verification**: Prior to delivering candidate setpoint corrections $\Delta\vec{I}$ to hardware, the surrogate evaluates downstream centroid offsets $\Delta\vec{x}_{\text{BPM}}$ and beam loss fractions. The `HELIOS-Safety` gatekeeper verifies these against operational loss ceilings before granting EPICS authorization.

2.2 Reinforcement Learning (RL) & Neural MPC Acceleration

1. **Surrogate-Accelerated Gymnasium Environment (`LinacGymEnv`)**: Executing numerical ray-tracing (ANL TRACK via Wine 64-bit) during continuous RL policy training requires approximately 1 s per step. For 10^5 training transitions, full simulation demands over 27 hours. Wrapping `raon`'s trained neural surrogate within `LinacGymEnv` reduces step latency to < 0.3 ms, executing 10^5 steps in **under 30 seconds**.

2. **Differentiable Physics and Model Predictive Control (Neural MPC)**: Because `synapticTrack` neural networks are differentiable PyTorch modules, analytical gradients $\nabla_{\vec{u}}\mathcal{L}(\vec{y}_{\text{pred}})$ are computed via backpropagation:

$$\vec{u}^* = \arg\min_{\vec{u}} \left(\|\vec{x}_{\text{BPM}}(\vec{u}) - \vec{x}_{\text{target}}\|^2 + \lambda \|\vec{u} - \vec{u}_0\|^2 \right) \tag{2}$$

The `NeuralMPCController` executes multi-step gradient descent directly through surrogate network weights prior to live hardware actuation.

3 Strategic Architectural Evaluation: Digital Twin Location

A foundational architectural question is whether the Digital Twin should be developed **separately** from HELIOS or **included within** HELIOS.

3.1 Evaluation of Architectural Options

3.1.1 Option A: Monolithic Repository (Twin Included Inside HELIOS)

- **Concept**: Ingest RAON lattice decks, TRACK input decks, dataset generators, and training scripts directly into the `helios` repository.
- **Critical Drawbacks**:
  1. *Confidentiality Requirements*: The governance document `raon/AGENTS.md` explicitly mandates that RAON lattice geometries and experimental calibration data remain private. HELIOS is architected as an open, publishable multi-agent system (modeled after LBNL Osprey). Incorporating proprietary RAON files would prevent open-source publication or external collaboration.
  2. *Computational Domain Pollution*: Generating 50,000 multi-particle datasets via cluster batch scripts is an offline HPC simulation task. Placing heavy offline data-generation pipelines inside an online real-time control-room operations repository introduces architectural antipatterns.
  3. *Repository Bloat*: Gigabytes of HDF5 tracking datasets and PyTorch checkpoints would degrade Git repository performance.

3.1.2 Option B: Fully Decoupled Microservice (Twin as a Remote Server)

- **Concept**: Host the Digital Twin on an independent server accessed via HTTP REST or gRPC APIs.
- **Critical Drawbacks**: Introduces socket latency, serialization overhead, and network partition risks. HELIOS deterministic safety interlocks require sub-5 ms pre-flight responses, which remote network hops cannot guarantee.

3.1.3 Option C: Hybrid Two-Tier Architecture (Plugin Provider Pattern) [Recommended]

- **Concept**:
  - **HELIOS** maintains the generic, open **Digital Twin Host Interface** (`helios.physics.twin`).
  - `synapticTrack` provides machine-neutral ML and surrogate contracts.
  - `raon` provides the facility-specific implementation and confidential assets (lattices, trained weights, PV bindings) as an optional in-process Python plugin.
- **Advantages**: Combines zero-latency in-process Python execution (< 1 ms) with strict IP/confidentiality separation and clean dependency management.

4 Implementation Blueprint

4.1 Step 1: Automated Environment Integration (`setup.sh`)

Configure `setup.sh` to dynamically export Python search paths for `synapticTrack` and `raon` when present:

```
# Link local simulation codes and facility assets if available
if [[ -d "/home/cspark/Work/simulation_codes-working/synapticTrack/src" ]]; then
    export PYTHONPATH="/home/cspark/Work/simulation_codes-working/synapticTrack/src:${PYTHONPATH}"
fi

if [[ -d "/home/cspark/Work/projects/raon/src" ]]; then
    export PYTHONPATH="/home/cspark/Work/projects/raon/src:${PYTHONPATH}"
fi
```

4.2 Step 2: Facility Plugin Loader in HELIOS

Implement the dynamic plugin loader in `helios/facilities/raon.py`:

```
from pathlib import Path
from typing import Optional
import logging

logger = logging.getLogger(__name__)

class RaonFacilityPlugin:
    """Plugin loader for RAON-specific surrogate models and lattice digital twin."""

    def __init__(self, models_root: Optional[Path] = None):
        self.models_root = models_root or Path("/home/cspark/Work/projects/raon/models")
        self._surrogate = None

    def load_lebt_surrogate(self):
        try:
            from raon_adapters.lebt.ml.surrogate import LEBTSurrogateModel
            ckpt = self.models_root / "lebt/orbit_corrections/legacy_50000/best_model.pt"
            if ckpt.is_file():
                self._surrogate = LEBTSurrogateModel.load(checkpoint_path=ckpt)
                logger.info("Successfully loaded RAON LEBT 50k surrogate into HELIOS.")
                return self._surrogate
        except ImportError:
            logger.warning("raon_adapters not found; using baseline analytical twin.")
        return None
```

4.3 Step 3: Surrogate-Driven RL Gymnasium Environment

Wire the surrogate into the continuous control environment:

```python
import gymnasium as gym

class LinacSurrogateEnv(gym.Env):
    """Sub-millisecond gymnasium environment powered by RAON neural surrogate."""

    def __init__(self, surrogate_model):
        self.surrogate = surrogate_model

    def step(self, action):
        # Action represents delta steerer currents constrained to 0.05 A DAC steps
        pred = self.surrogate.predict(action)
        reward = -float(pred.physical[..., 0]**2 + pred.physical[..., 1]**2)
        terminated = bool(pred.physical[..., -1] > 1.0)  # Loss check
        return pred.physical, reward, terminated, False, {}
```

5 Summary of Recommendations

1. **Repository Decoupling**: Maintain `synapticTrack` as open and machine-neutral. Retain `raon` as a private facility repository holding proprietary lattices and datasets. Maintain `helios` as an open multi-agent framework.

2. **In-Process Plugin Linking**: Link repositories through local Python package imports rather than remote microservices, ensuring sub-millisecond execution latency essential for machine protection.

3. **Surrogate Acceleration for RL**: Leverage `raon`'s pre-trained 50,000-sample surrogate to accelerate RL policy training by over $1,000\times$ relative to live TRACK particle-tracking simulations.